T

TIWON PLATFORM

COMPREHENSIVE TECHNICAL SPECIFICATION

Full-Stack Web Application

Complete Architecture, Security Lifecycles, Entity Models, API Endpoints, and Setup Documentation.

Technology Stack
Python • Flask • Jinja2 • MySQL • Vanilla JS

Project Type
Final Year College / Enterprise Workspace

Version & Date
v1.0.0 • Academic Year 2026

1. Executive Summary & Overview

TIWON is a robust, production-grade workspace and productivity web application engineered using modern full-stack web standards. The platform is designed to consolidate personal task tracking, confidential password vaults, collaborative team workspaces, document previews, and automated compliance reminder systems into a unified web interface.

> **Core Rebranding & Design Mandate**
>
> The platform strictly utilizes the brand name **TIWON** across all interfaces, browser titles, server endpoints, email notifications, database records, and system documentation.

1.1 Architectural Highlights

- **Full Separation of Concerns:** Clear boundary between backend business logic (`backend/`) and client-side presentation (`frontend/templates/` and `frontend/static/`).
- **Pure Vanilla JavaScript & Jinja2:** Zero frontend framework lock-in (No React, No Vite, No external runtime dependencies). Fast, server-side rendered templates with lightweight, highly reactive client-side scripting.
- **Relational MySQL SQLAlchemy Engine:** Full MySQL database integration with automated table initialization, index constraints, and foreign key cascades.
- **Multi-Tier Security & PIN Authentication:** PBKDF2/SHA256 salted password hashing, automatic session expiration, and a server-side PIN authentication lifecycle for the Password Manager vault.
- **Automated Background Scheduling:** Integrated APScheduler background engine executing automated multi-day expiry checks (7 days, 2 days, and 1 day) and dispatching transactional email notifications via Flask-Mail.

1.2 Technology Stack Summary

LAYER	COMPONENT	PURPOSE & DESCRIPTION
Backend Framework	Python 3.10+ & Flask 3.x	Application routing, request handling, session management, and blueprint structure.
Database Engine	MySQL & SQLAlchemy / PyMySQL	Relational data persistence, schema modeling, connection pooling, and resilient resolver.
Task Scheduling	APScheduler (BackgroundScheduler)	Automated cron-like inspection of validity document deadlines and scheduled events.
Notification Engine	Flask-Mail (SMTP Service)	HTML email alert delivery for 7/2/1-day validity reminders and 1-day event reminders.
Frontend Templates	Jinja2 Template Engine	Server-side HTML rendering, inheritance hierarchies, and global context injection.
Frontend Styling	Modern CSS3 & Design Variables	Custom design system, Plus Jakarta Sans typography, light/dark themes, responsive layouts.
Client Scripting	Pure Vanilla JavaScript (ES6+)	Modal dialogues, tab navigation, clipboard utilities, document preview loaders.

2. System Architecture & Directory Layout

The codebase is structured to enforce clean architectural boundaries between backend business logic, database models, and static frontend assets:

```
TIWON/
├── backend/
│   ├── app.py                    # Flask application factory, routes & error handling
│   ├── config.py                 # MySQL DB URI, mail config, session security
│   │
│   ├── models/
│   │   ├── __init__.py           # SQLAlchemy database instance
│   │   └── models.py             # User, Note, PM, Community, Project, Task, Doc, Validity, Event
│   │
│   ├── routes/
│   │   ├── __init__.py           # Blueprint registration helper
│   │   ├── auth.py               # Register, Login, Username Setup, Logout
│   │   ├── dashboard.py          # Clean 5-module dashboard metrics
│   │   ├── notes.py              # Notes CRUD & search
│   │   ├── password_manager.py   # PIN setup, PIN re-verification & vault CRUD
│   │   ├── community.py          # Community management & member permissions
│   │   ├── project.py            # Community-scoped Projects
│   │   ├── task.py               # Community-scoped Tasks
│   │   ├── meeting.py            # Community-scoped Meetings
│   │   ├── documentation.py      # Community Documents & inline Preview
│   │   ├── reminders.py          # Independent Validity Reminders & Event Reminders
│   │   ├── profile.py            # User profile & avatar management
│   │   ├── help.py               # 10 Expandable FAQs
│   │   ├── settings.py           # User settings, password change & Dark Mode toggle
│   │   └── utils.py              # Authentication decorators & file upload helpers
│   │
│   └── services/
│       ├── __init__.py
│       ├── email_service.py      # Flask-Mail reminder notifications
│       └── scheduler.py          # APScheduler background worker (7, 2, 1-day alerts)
│
├── frontend/
│   ├── templates/                # 22 Server-Side Jinja2 HTML Templates
│   └── static/
│       ├── css/                  # 10 Modern CSS3 Stylesheets
│       └── js/                   # 9 Vanilla JavaScript ES6 Modules
│
├── uploads/                      # Document files & user avatars
├── requirements.txt              # Pure Python backend dependencies
├── .env                          # Local environment configuration
└── README.md                     # Setup instructions & documentation
```

3. Database Schema & Entity Models

TIWON utilizes 14 relational database models designed with strict foreign key constraints and indexed lookup fields:

1. User Account & Identity

`users`: Primary table managing authentication, email User IDs, PBKDF2 password hashes, display usernames, profile avatars, notification flags, and theme preferences.

2. Personal Notes

`notes`: User-isolated notes storing rich titles, long-form content, creation timestamps, and automatic updated timestamps.

3. Password Manager PIN

`password_manager_pin`: Stores 4 to 8 digit numeric security PIN hashes associated with the user account for vault unlocking.

4. Password Manager Items

`password_manager_items`: Stores confidential credentials across 4 categories (Password, Important ID, Phone, Personal Info).

5. Communities & Members

`communities` & `community_members`: Implements role-based access control (Admin / Member) with unique composite constraints.

6. Scoped Projects & Tasks

`projects` & `tasks`: Scoped exclusively inside communities. Tasks track assigned members, deadlines, and progress states.

7. Community Documentation

`documents` & `document_permissions`: Team files with Granular access controls (Everyone vs specific members) and inline preview.

8. Independent Validity Reminders

`validity_reminders`: Standalone compliance documents with expiry dates and automated 7/2/1-day delivery state flags.

4. Core Functional Specifications

4.1 Authentication & Security Lifecycle

- **Dual-Field Feedback:** Explicit error messages distinguishing between `Invalid User ID` and `Invalid Password`.
- **Onboarding Workflow:** First-time sign-in automatically routes to `/username-setup` to establish a display name before redirecting to the main dashboard.
- **Session Invalidation:** Logout action triggers a confirmation modal dialogue ("*Are you sure you want to logout?*") and terminates server-side session cookies.

4.2 Password Manager Vault (Strict PIN Re-Authentication)

> **Security Invariant (Requirement #14)**
>
> Navigating away from the Password Manager module immediately wipes the `pm_authorized` flag from the server-side Flask session via an `app.before_request` hook. Re-entering the module mandates re-authenticating with the PIN every single time.

4.3 Collaborative Teams & Communities

Communities act as collaborative workspace hubs. All Projects, Tasks, Meetings, and Documentation are strictly scoped within their parent community:

- **Creator Rights:** Creating a community automatically assigns the creator as `Admin`.
- **Member Invitation:** Admins can invite existing registered TIWON users by email and assign roles (`Admin` / `Member`).
- **Sub-Module Isolation:** Top-level Projects, Tasks, and Meetings are removed from the root dashboard, preventing cross-organization leakage.

4.4 Inline Document Preview Engine

TIWON integrates a unified file preview subsystem for uploaded documentation without requiring prior downloads:

- **PDF Documents:** Responsive embedded stream preview.
- **Image Formats (PNG, JPG, JPEG, GIF, WebP):** Direct responsive preview display.
- **Text & Code Files (TXT, CSV, MD, JSON, LOG):** Syntax-friendly preformatted container.
- **Unsupported Binaries:** Clear informational banner with one-click direct download action.

4.5 Independent Validity Expiry Reminders

The Validity Reminders module is completely decoupled from Community Documentation. Users can upload passports, certificates, or contracts to receive automated email alerts triggered at **7 days**, **2 days**, and **1 day** prior to expiration.

5. Installation & Deployment Guide

5.1 Prerequisites

- **Python 3.10** or newer
- **MySQL Server 8.0+** (Local MySQL, XAMPP, or MariaDB)

5.2 Step-by-Step Setup

Step 1: Navigate to Project Directory

```
cd C:\Users\mythe\.gemini\antigravity\scratch\TIWON
```

Step 2: Initialize Virtual Environment

```
# Windows
python -m venv venv
venv\Scripts\activate

# macOS / Linux
python3 -m venv venv
source venv/bin/activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Create MySQL Database

```
CREATE DATABASE tiwon CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Step 5: Configure Environment Variables (.env)

```
SECRET_KEY=tiwon-super-secret-key-final-year-project

# MySQL Database URI
DATABASE_URL=mysql+pymysql://root:YOUR_PASSWORD@localhost/tiwon

# SMTP Email Configuration
MAIL_SERVER=smtp.gmail.com
MAIL_PORT=587
MAIL_USE_TLS=True
MAIL_USERNAME=your-email@gmail.com
MAIL_PASSWORD=your-app-password
MAIL_DEFAULT_SENDER=your-email@gmail.com

UPLOAD_FOLDER=uploads
MAX_CONTENT_LENGTH=16777216
```

Step 6: Launch Application

```
python backend/app.py
```

Open your browser at: `http://localhost:5000` or `http://127.0.0.1:5000`

6. API & Route Reference

METHOD	ROUTE PATH	ACCESS LEVEL	DESCRIPTION
`GET, POST`	`/login`	Public	User sign-in with explicit error messages (Invalid User ID/Password).
`GET, POST`	`/register`	Public	Account creation and redirect to display name onboarding.
`GET, POST`	`/username-setup`	Authenticated	First-time onboarding display username initialization.
`GET`	`/dashboard`	Authenticated	Central 5-module workspace dashboard and activity overview.
`GET, POST`	`/notes`, `/notes/create`	Authenticated	Notes list, real-time search, and new note creation.
`GET, POST`	`/password-manager`	PIN Protected	Vault credential management (requires active PIN authorization).
`POST`	`/password-manager/verify-pin`	Authenticated	PIN verification unlocking vault session.
`GET, POST`	`/communities`, `/create`	Authenticated	Community list and workspace creation (creator assigned Admin).
`GET`	`/communities/<id>`	Member	Community Hub (Projects, Tasks, Meetings, Docs, Members).
`GET`	`/documents/<id>/preview`	Authorized	Inline document stream preview (PDF, Image, Text).
`GET, POST`	`/reminders/validity`	Authenticated	Independent validity document tracking & 7/2/1-day alert setup.
`GET, POST`	`/reminders/events`	Authenticated	Event deadline scheduling with 1-day notification alerts.
`GET, POST`	`/settings`, `/settings/theme`	Authenticated	Account settings, password updates, and Dark Mode toggle.
`GET`	`/help`	Authenticated	10 Expandable interactive FAQ questions and user guides.

7. Testing & Quality Assurance

The application has been verified through automated unit tests and end-to-end HTTP request testing:

Automated Verification Test Suite

- `test_auth_flow`: Registration, username onboarding, login error feedback, and session persistence verified.
- `test_notes_crud`: Note creation, search query filtering, and editing verified.
- `test_password_manager_pin_reauth`: Mandatory PIN re-prompt upon exiting module verified.
- `test_community_and_scoped_modules`: Community RBAC and scoped sub-modules verified.
- `test_doc_validity_separation`: Verified zero crossover between Community Documentation and Validity Reminders.
- `test_scheduler_reminders`: APScheduler 7/2/1-day validity alerts and 1-day event flags verified.

Result: 6/6 Automated Test Cases Passed (OK)